

# Changelog

## 1.4.0 (2024-06-11)

- Feature: Improve PHP 8.4+ support by avoiding implicitly nullable type declarations. (#179 by @clue)
- Feature: Full PHP 8.3 compatibility. (#172 by @clue)
- Fix: Fix `drain` event of `ThroughStream` to handle potential race condition. (#171 by @clue)

## 1.3.0 (2023-06-16)

- Feature: Full PHP 8.1 and PHP 8.2 compatibility. (#160 by @SimonFrings, #165 by @clue and #169 by @WyriHaximus)
- Feature: Avoid unneeded syscall when creating non-blocking `DuplexResourceStream`. (#164 by @clue)
- Minor documentation improvements. (#161 by @mrsimonbennett, #162 by @SimonFrings and #166 by @nhedger)
- Improve test suite and project setup and report failed assertions. (#168 and #170 by @clue and #163 by @SimonFrings)

## 1.2.0 (2021-07-11)

A major new feature release, see **release announcement**.

- Feature: Simplify usage by supporting new default loop. (#159 by @clue)

```
// old (still supported)
$stream = new ReadableResourceStream($resource, $loop);
$stream = new WritableResourceStream($resource, $loop);
$stream = new DuplexResourceStream($resource, $loop);

// new (using default loop)
$stream = new ReadableResourceStream($resource);
$stream = new WritableResourceStream($resource);
$stream = new DuplexResourceStream($resource);
```

- Improve test suite, use GitHub actions for continuous integration (CI), update PHPUnit config, run tests on PHP 8 and add full core team to the license. (#153, #156 and #157 by @SimonFrings and #154 by @WyriHaximus)

## 1.1.1 (2020-05-04)

- Fix: Fix faulty write buffer behavior when sending large data chunks over TLS (Mac OS X only). (#150 by @clue)

- Minor code style improvements to fix phpstan analysis warnings and add `.gitattributes` to exclude dev files from exports. (#140 by @flow-control and #144 by @reedy)
- Improve test suite to run tests on PHP 7.4 and simplify test matrix. (#147 by @clue)

### 1.1.0 (2019-01-01)

- Improvement: Increase performance by optimizing global function and constant look ups. (#137 by @WyriHaximus)
- Travis: Test against PHP 7.3. (#138 by @WyriHaximus)
- Fix: Ignore empty reads. (#139 by @WyriHaximus)

### 1.0.0 (2018-07-11)

- First stable LTS release, now following SemVer. We'd like to emphasize that this component is production ready and battle-tested. We plan to support all long-term support (LTS) releases for at least 24 months, so you have a rock-solid foundation to build on top of.

Contains no other changes, so it's actually fully compatible with the v0.7.7 release.

### 0.7.7 (2018-01-19)

- Improve test suite by fixing forward compatibility with upcoming Event-Loop releases, avoid risky tests and add test group to skip integration tests relying on internet connection and apply appropriate test timeouts. (#128, #131 and #132 by @clue)

### 0.7.6 (2017-12-21)

- Fix: Work around reading from unbuffered pipe stream in legacy PHP < 5.4.28 and PHP < 5.5.12 (#126 by @clue)
- Improve test suite by simplifying test bootstrapping logic via Composer and test against PHP 7.2 (#127 by @clue and #124 by @carusogabriel)

### 0.7.5 (2017-11-20)

- Fix: Ignore excessive `fopen()` mode flags for `WritableResourceStream` (#119 by @clue)
- Fix: Fix forward compatibility with upcoming EventLoop releases (#121 by @clue)
- Restructure examples to ease getting started (#123 by @clue)

- Improve test suite by adding forward compatibility with PHPUnit 6 and ignore Mac OS X test failures for now until Travis tests work again (#122 by @gabriel-caruso and #120 by @clue)

#### 0.7.4 (2017-10-11)

- Fix: Remove event listeners from `CompositeStream` once closed and remove undocumented left-over `close` event argument (#116 by @clue)
- Minor documentation improvements: Fix wrong class name in example, fix typos in README and fix forward compatibility with upcoming EventLoop releases in example (#113 by @docteurklein and #114 and #115 by @clue)
- Improve test suite by running against Mac OS X on Travis (#112 by @clue)

#### 0.7.3 (2017-08-05)

- Improvement: Support Événement 3.0 a long side 2.0 and 1.0 (#108 by @WyriHaximus)
- Readme: Corrected loop initialization in usage example (#109 by @pulyavin)
- Travis: Lock linux distribution preventing future builds from breaking (#110 by @clue)

#### 0.7.2 (2017-06-15)

- Bug fix: WritableResourceStream: Close the underlying stream when closing the stream. (#107 by @WyriHaximus)

#### 0.7.1 (2017-05-20)

- Feature: Add optional `$writeChunkSize` parameter to limit maximum number of bytes to write at once. (#105 by @clue)
- ```
$stream = new WritableResourceStream(STDOUT, $loop, null, 8192);
```
- Ignore HHVM test failures for now until Travis tests work again (#106 by @clue)

#### 0.7.0 (2017-05-04)

- Removed / BC break: Remove deprecated and unneeded functionality (#45, #87, #90, #91 and #93 by @clue)
  - Remove deprecated `Stream` class, use `DuplexResourceStream` instead (#87 by @clue)

- Remove public `$buffer` property, use new constructor parameters instead (#91 by @clue)
- Remove public `$stream` property from all resource streams (#90 by @clue)
- Remove undocumented and now unused `ReadableStream` and `WritableStream` (#93 by @clue)
- Remove `BufferedSink` (#45 by @clue)
- Feature / BC break: Simplify `ThroughStream` by using data callback instead of inheritance. It is now a direct implementation of `DuplexStreamInterface`. (#88 and #89 by @clue)
 

```
$through = new ThroughStream(function ($data) {
    return json_encode($data) . PHP_EOL;
});
$through->on('data', $this->expectCallableOnceWith("[2, true]\n"));

$through->write(array(2, true));
```
- Feature / BC break: The `CompositeStream` starts closed if either side is already closed and forwards pause to pipe source on first write attempt. (#96 and #103 by @clue)
 

If either side of the composite stream closes, it will also close the other side. We now also ensure that if either side is already closed during instantiation, it will also close the other side.
- BC break: Mark all classes as `final` and mark internal API as `private` to discourage inheritance (#95 and #99 by @clue)
- Feature / BC break: Only emit `error` event for fatal errors (#92 by @clue)
 

The `error` event was previously also allowed to be emitted for non-fatal errors, but our implementations actually only ever emitted this as a fatal error and then closed the stream.
- Feature: Explicitly allow custom events and exclude any semantics (#97 by @clue)
- Strict definition for event callback functions (#101 by @clue)
- Support legacy PHP 5.3 through PHP 7.1 and HHVM and improve usage documentation (#100 and #102 by @clue)
- Actually require all dependencies so this is self-contained and improve forward compatibility with EventLoop v1.0 and v0.5 (#94 and #98 by @clue)

## 0.6.0 (2017-03-26)

- Feature / Fix / BC break: Add `DuplexResourceStream` and deprecate `Stream` (#85 by @clue)

```
// old (does still work for BC reasons)
$stream = new Stream($connection, $loop);
```

```
// new
$stream = new DuplexResourceStream($connection, $loop);
```

Note that the `DuplexResourceStream` now rejects read-only or write-only streams, so this may affect BC. If you want a read-only or write-only resource, use `ReadableResourceStream` or `WritableResourceStream` instead of `DuplexResourceStream`.

BC note: This class was previously called `Stream`. The `Stream` class still exists for BC reasons and will be removed in future versions of this package.

- Feature / BC break: Add `WritableResourceStream` (previously called `Buffer`) (#84 by @clue)

```
// old
$stream = new Buffer(STDOUT, $loop);
```

```
// new
$stream = new WritableResourceStream(STDOUT, $loop);
```

- Feature: Add `ReadableResourceStream` (#83 by @clue)

```
$stream = new ReadableResourceStream(STDIN, $loop);
```

- Fix / BC Break: Enforce using non-blocking I/O (#46 by @clue)

BC note: This is known to affect process pipes on Windows which do not support non-blocking I/O and could thus block the whole EventLoop previously.

- Feature / Fix / BC break: Consistent semantics for `DuplexStreamInterface::end()` to ensure it SHOULD also end readable side (#86 by @clue)
- Fix: Do not use unbuffered reads on pipe streams for legacy PHP < 5.4 (#80 by @clue)

## 0.5.0 (2017-03-08)

- Feature / BC break: Consistent `end` event semantics (EOF) (#70 by @clue)

The `end` event will now only be emitted for a *successful* end, not if the stream closes due to an unrecoverable `error` event or if you call `close()`

explicitly. If you want to detect when the stream closes (terminates), use the `close` event instead.

- BC break: Remove custom (undocumented) `full-drain` event from `Buffer` (#63 and #68 by @clue)

The `full-drain` event was undocumented and mostly used internally. Relying on this event has attracted some low-quality code in the past, so we've removed this from the public API in order to work out a better solution instead. If you want to detect when the buffer finishes flushing data to the stream, you may want to look into its `end()` method or the `close` event instead.

- Feature / BC break: Consistent event semantics and documentation, explicitly state *when* events will be emitted and *which* arguments they receive. (#73 and #69 by @clue)

The documentation now explicitly defines each event and its arguments. Custom events and event arguments are still supported. Most notably, all defined events only receive inherently required event arguments and no longer transmit the instance they are emitted on for consistency and performance reasons.

```
// old (inconsistent and not supported by all implementations)
$stream->on('data', function ($data, $stream) {
  // process $data
});

// new (consistent throughout the whole ecosystem)
$stream->on('data', function ($data) use ($stream) {
  // process $data
});
```

This mostly adds documentation (and thus some stricter, consistent definitions) for the existing behavior, it does NOT define any major changes otherwise. Most existing code should be compatible with these changes, unless it relied on some undocumented/unintended semantics.

- Feature / BC break: Consistent method semantics and documentation (#72 by @clue)

This mostly adds documentation (and thus some stricter, consistent definitions) for the existing behavior, it does NOT define any major changes otherwise. Most existing code should be compatible with these changes, unless it relied on some undocumented/unintended semantics.

- Feature: Consistent `pipe()` semantics for closed and closing streams (#71 from @clue)

The source stream will now always be paused via `pause()` when the destination stream closes. Also, properly stop piping if the source stream closes and remove all event forwarding.

- Improve test suite by adding PHPUnit to `require-dev` and improving coverage. (#74 and #75 by @clue, #66 by @nawarian)

#### 0.4.6 (2017-01-25)

- Feature: The `Buffer` can now be injected into the `Stream` (or be used standalone) (#62 by @clue)
- Fix: Forward `close` event only once for `CompositeStream` and `ThroughStream` (#60 by @clue)
- Fix: Consistent `close` event behavior for `Buffer` (#61 by @clue)

#### 0.4.5 (2016-11-13)

- Feature: Support setting read buffer size to `null` (infinite) (#42 by @clue)
- Fix: Do not emit `full-drain` event if `Buffer` is closed during `drain` event (#55 by @clue)
- Vastly improved performance by factor of 10x to 20x. Raise default buffer sizes to 64 KiB and simplify and improve error handling and unneeded function calls. (#53, #55, #56 by @clue)

#### 0.4.4 (2016-08-22)

- Bug fix: Emit `error` event and close `Stream` when accessing the underlying stream resource fails with a permanent error. (#52 and #40 by @clue, #25 by @lysenkobv)
- Bug fix: Do not emit empty `data` event if nothing has been read (stream reached EOF) (#39 by @clue)
- Bug fix: Ignore empty writes to `Buffer` (#51 by @clue)
- Add benchmarking script to measure throughput in CI (#41 by @clue)

#### 0.4.3 (2015-10-07)

- Bug fix: Read buffer to 0 fixes error with libevent and large quantity of I/O (@mbonneau)
- Bug fix: No double-write during drain call (@arnaud-lb)
- Bug fix: Support HHVM (@clue)
- Adjust compatibility to 5.3 (@clue)

#### **0.4.2 (2014-09-09)**

- Added DuplexStreamInterface
- Stream sets stream resources to non-blocking
- Fixed potential race condition in pipe

#### **0.4.1 (2014-04-13)**

- Bug fix: v0.3.4 changes merged for v0.4.1

#### **0.3.4 (2014-03-30)**

- Bug fix: [Stream] Fixed 100% CPU spike from non-empty write buffer on closed stream

#### **0.4.0 (2014-02-02)**

- BC break: Bump minimum PHP version to PHP 5.4, remove 5.3 specific hacks
- BC break: Update to Evenement 2.0
- Dependency: Autoloading and filesystem structure now PSR-4 instead of PSR-0

#### **0.3.3 (2013-07-08)**

- Bug fix: [Stream] Correctly detect closed connections

#### **0.3.2 (2013-05-10)**

- Bug fix: [Stream] Make sure CompositeStream is closed properly

#### **0.3.1 (2013-04-21)**

- Bug fix: [Stream] Allow any ReadableStreamInterface on BufferedSink::createPromise()

#### **0.3.0 (2013-04-14)**

- Feature: [Stream] Factory method for BufferedSink

#### **0.2.6 (2012-12-26)**

- Version bump

#### **0.2.5 (2012-11-26)**

- Feature: Make BufferedSink trigger progress events on the promise (@jsor)

#### **0.2.4 (2012-11-18)**

- Feature: Added ThroughStream, CompositeStream, ReadableStream and WritableStream
- Feature: Added BufferedSink

#### **0.2.3 (2012-11-14)**

- Version bump

#### **0.2.2 (2012-10-28)**

- Version bump

#### **0.2.1 (2012-10-14)**

- Bug fix: Check for EOF in `Buffer::write()`

#### **0.2.0 (2012-09-10)**

- Version bump

#### **0.1.1 (2012-07-12)**

- Bug fix: Testing and functional against PHP  $\geq 5.3.3$  and  $\leq 5.3.8$

#### **0.1.0 (2012-07-11)**

- First tagged release